

04 · REST autenticado con OAuth2 (client_credentials)

OAuth2 `client_credentials` es el flujo **máquina-a-máquina** (sin usuario que autorice nada en un navegador):

1. `POST /api/oauth/token` con `client_id` + `client_secret` -> devuelve un `access_token` (tipo *Bearer*) que **expira** (`expires_in: 3600` segundos).
2. Cada petición a la API lleva `Authorization: Bearer <access_token>`.
3. Cuando el token caduca, se vuelve a pedir otro.

Cliente demo: `scraper-demo` / `scraper-demo-secret` (ver `tienda-virtual/.env.example`; se crean más en `/admin/oauth-clients`). Los scopes son `read` (para los `GET`) y `write` (para mutaciones); el cliente demo tiene ambos.

```
import csv
import os
import time

import requests

BASE = "http://localhost:3000"
CLIENT_ID = "scraper-demo"
CLIENT_SECRET = "scraper-demo-secret"
TIMEOUT = 30
```

Pedir un token

El endpoint acepta `application/x-www-form-urlencoded` (lo que hace `requests` con `data=`), `JSON` o `Authorization: Basic`.

```
r = requests.post(
    f"{BASE}/api/oauth/token",
    data={
        "grant_type": "client_credentials",
        "client_id": CLIENT_ID,
        "client_secret": CLIENT_SECRET,
    },
    timeout=TIMEOUT,
)
r.raise_for_status()
token_resp = r.json()
print({k: (v[:28] + "...") if k == "access_token" else v for k, v in
      token_resp.items()})

{'access_token': 'eyJhbGciOiJIUzI1NiJ9.eyJzY29...', 'token_type': 'Bearer',
 'expires_in': 3600, 'scope': 'read write'}
```

Credencial equivocada: 401 invalid_client

```
r = requests.post(
    f"{BASE}/api/oauth/token",
    data={
        "grant_type": "client_credentials",
        "client_id": CLIENT_ID,
        "client_secret": "secreto-incorrecto",
    },
    timeout=TIMEOUT,
)
print(r.status_code, r.json())
```

```
401 {'error': 'invalid_client', 'error_description': 'client_id o client_secret
invalidos.'}
```

Un cliente que renueva el token solo

`ClienteOAuth` pide el token la primera vez, lo cachea y lo renueva automáticamente unos segundos antes de que caduque. Así el bucle de scraping no se preocupa por la expiración.

```
class ClienteOAuth:
    def __init__(self, base, client_id, client_secret):
        self.base = base
        self.client_id = client_id
        self.client_secret = client_secret
        self.sesion = requests.Session()
        self.sesion.headers["User-Agent"] = "MineriaWeb-2026-2/1.0"
        self._token = None
        self._expira_en = 0.0

    def _renovar(self):
        r = requests.post(
            f"{self.base}/api/oauth/token",
            data={
                "grant_type": "client_credentials",
                "client_id": self.client_id,
                "client_secret": self.client_secret,
            },
            timeout=TIMEOUT,
        )
        r.raise_for_status()
        d = r.json()
        self._token = d["access_token"]
        # renovamos 60 s antes del vencimiento real, con margen de seguridad
        self._expira_en = time.monotonic() + d["expires_in"] - 60
        print(f" [token renovado, válido ~{d['expires_in']} s]")

    def _cabecera_auth(self):
        if self._token is None or time.monotonic() >= self._expira_en:
            self._renovar()
        return {"Authorization": f"Bearer {self._token}"}

    def get(self, path, **params):
        r = self.sesion.get(
            f"{self.base}/{path}",
```

```

        headers=self._cabecera_auth(),
        params=params,
        timeout=TIMEOUT,
    )
    r.raise_for_status()
    return r.json()

```

```

api = ClienteOAuth(BASE, CLIENT_ID, CLIENT_SECRET)
print("Prueba:", api.get("/api/productos", page=1, pageSize=1)["pageInfo"])

```

[token renovado, válido ~3600 s]
 Prueba: {'page': 1, 'pageSize': 1, 'total': 90, 'totalPages': 90}

Scraping paginado con el token Bearer

Bajamos el listado de **órdenes**. Cada orden ya trae su **cliente** y sus **items** embebidos (con el **producto** de cada ítem).

```

def descargar_todo(path, page_size=50, **filtros):
    items = []
    page = 1
    while True:
        payload = api.get(path, page=page, pageSize=page_size, **filtros)
        items.extend(payload["items"])
        info = payload["pageInfo"]
        print(f" {path} página {info['page']}>2}/{info['totalPages']} (total
{len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

ordenes = descargar_todo("/api/ordenes")
print(f"\n{len(ordenes)} órdenes")

```

```

/api/ordenes página 1/6 (total 50/300)
/api/ordenes página 2/6 (total 100/300)
/api/ordenes página 3/6 (total 150/300)
/api/ordenes página 4/6 (total 200/300)
/api/ordenes página 5/6 (total 250/300)
/api/ordenes página 6/6 (total 300/300)

```

300 órdenes

Guardar en data/rest_ordenes.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)

ruta_ordenes = os.path.join(DATA_DIR, "rest_ordenes.csv")
with open(ruta_ordenes, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "numero", "fecha", "cliente_id", "cliente_nombre",
"cliente_pais", "num_items", "sub_total", "igv", "total"],
    )

```

```

writer.writeheader()
for o in ordenes:
    cli = o["cliente"]
    writer.writerow({
        "id": o["id"],
        "numero": o["numero"],
        "fecha": o["fecha"],
        "cliente_id": cli["id"],
        "cliente_nombre": f"{cli['nombre']} {cli['apellidos']}",
        "cliente_pais": cli["pais"],
        "num_items": len(o["items"]),
        "sub_total": o["subTotal"],
        "igv": o["igv"],
        "total": o["total"],
    })

print("Guardado:", ruta_ordenes)

```

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_ordenes.csv

Resumiendo la parte REST: **01** primer contacto, **02** paginación, **03** la API key a fondo, **04** OAuth2. En los dos notebooks siguientes cambiamos de estilo de API: GraphQL.